- Object-Oriented Programming is a programming paradigm based on the concept of "objects", which can contain data, in the form of fields or attributes, and code, in the form of procedures or methods.

- By utilizing these concepts, Object-Oriented Programming helps in creating modular, maintainable, and scalable code. It promotes a clear separation of concerns and allows for easier collaboration in larger software projects.

- OOP have four concepts :

Class and Object

1-Encapsulation

2- Inheritance

3-Polymorphism

4-Abstraction

Class and Object

1. A class is like a blueprint for creating objects. It defines the properties (attributes) and behaviors (methods) that all objects of that type will have.

2. instance of a class. It represents a specific instance of the class, with its own unique set of data and behavior.

```
class Human {
  String? name;
  int? age;
  double? height;
  double? wight;
}

void main() {
  Human human = Human();

  human.name = 'Mohamed';
  human.age = 32;
  human.height = 169.1;
  human.wight = 70.5;
}
```

- methods within a class are defined similarly to regular functions, but they are indented within the class definition

```
class Human {
  String? name;
  int? age;
  double? height;
  double? wight;

  void run()
  {
    print("Running....");
  }
}

void main() {
  Human human = Human();

  human.name = 'Mohamed';
  human.age = 32;
  human.height = 169.1;
  human.wight = 70.5;
  human.run();
}
```


```
class Human {
  String? name;
  int? age;
  double? height;
  double? wight;
```

constructors

- Named constructors are a type of constructor that allows you to define multiple constructors for a class, each with a unique name. Named constructors are particularly useful when you want to provide different ways to create instances of a class with different sets of parameters or initialization logic.

```
void main() {

Circle circle1 =Circle(radius: 5, x: 1, y: 2);
Circle circle2 =Circle.origin(radius: 5, );

print(circle1.draw());
print(circle2.draw());
}
class Circle {
  double radius;
  int?x ;
  int?y;

  String draw ()=>"Drawing the circle with x = $x and y = $y with radius: $radius";

  Circle ({required this.radius, required this.x, required this.y});

  Circle.origin({required this.radius}){
    x = 0;
    y=0;
  }
}
```

```
  int year;
  bool isRunning;

  // Constructor
  Car(String make, String model, int year) {
    // Attribute initialization
    this.make = make;
    this.model = model;
    this.year = year;
    isRunning = false;
  }

  // Methods...
}
```

```
  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }
}

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2);
  print(human.name);



  human.run();
}
```

- Encapsulation involves hiding the internal state of an object, including its attributes, to prevent direct access and manipulation from outside the class.

```dart
class Human {
  String? name;
  int? age;
  int _numberOfArms =2;
  double? height;
  double? weight;

  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }
}
```

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);
  human._numberOfArms =10;
  print(human.name);


  human.run();
}
```

- By encapsulating attributes and providing controlled access through methods, you can ensure that the object's data remains consistent and valid.

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);

  human.setNumberOfArms(1);

  print(human.getNumberOfArms());

}
```

```dart
class Human {
  String? name;
  int? age;
  int _numberOfArms =2;
  double? height;
  double? weight;

  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }

  void setNumberOfArms(int numberOfArms)
  {
    if(numberOfArms<=2&&numberOfArms>=0)
      {
        _numberOfArms = numberOfArms;
      }
  }
  int getNumberOfArms()
  {
    return _numberOfArms;
  }
}
```

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);

  human.setNumberOfArms = 1;

  print(human.getNumberOfArms());

}
```

```
class Human {
  String? name;
  int? age;
  int _numberOfArms =2;
  double? height;
  double? weight;

  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }

  set setNumberOfArms(int numberOfArms)
  {
    if(numberOfArms<=2&&numberOfArms>=0)
      {
        _numberOfArms = numberOfArms;
      }

  int getNumberOfArms()
  {
    return _numberOfArms;
  }
}
```

```
  //Normal function
  int getNumberOfArms()
  {
    return _numberOfArms;
  }
//Arrow Function
  int numbersOfArms () => _numberOfArms;
}
```

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);

  human.setNumberOfArms = 1;

  print(human.getNumbersOfArms);

}
```

```dart
  set setNumberOfArms(int numberOfArms) {
    if (numberOfArms <= 2 && numberOfArms >= 0) {
      _numberOfArms = numberOfArms;
    }
  }

  int get getNumbersOfArms=> _numberOfArms;


}
```

- Allows one class to inherit properties and behaviors (methods) from another class. inheritance enables code reuse and promotes a hierarchical organization of classes.

- An abstract class is a class that cannot be instantiated directly. It serves as a template or blueprint for other classes and may contain one or more abstract methods. Abstract classes are meant to be subclassed, and they can provide default implementations for some methods.